

Standard Operating Procedure (SOP): Arctos Robotic Arm — CANable Bring-Up, MKS Servo42D CAN Protocol, and ROS 2 Joint Control (Joint B)

Assembled from project debugging logs, source-code inspection,
and empirical bench testing

August 2026

Contents

1	Introduction	3
1.1	CAN Bus, in one paragraph	3
1.2	Gearboxes and Closed-Loop Steppers, in one paragraph	3
2	Hardware Overview	4
2.1	Joint / CAN ID Map	4
2.2	Joint B Motor / Controller Specifications	4
2.3	CANable 2 Adapter	4
2.4	Wiring Diagram	5
2.5	Bus Termination Check (power off)	5
3	Bringing Up the CAN Interface on Ubuntu	6
3.1	Step 1: Confirm the USB device is visible	6
3.2	Step 2: Find the serial device	6
3.3	Step 3: Load the CAN driver if needed	6
3.4	Step 4: Clean up any old session	6
3.5	Step 5: Bind the adapter to a CAN interface	6
3.6	Step 6: Bring the link up at the correct bitrate	7
3.7	Step 7: Verify	7
3.8	Shutdown procedure	7
4	MKS Servo42D Panel Configuration	8
5	The MKS CAN Protocol as Used by Arctos	9
5.1	Frame Structure and Checksum	9
5.2	Command Reference	9
5.3	Enable / Disable / Query	10
5.4	Reading the Encoder	10
5.5	Motion Commands	11

6	From Raw Encoder Counts to Joint Degrees	13
7	Safety Commands	15
8	A Safe First Motion Test	16
8.1	Procedure	16
8.2	Script: <code>joint_b_micro_move_test.py</code>	16
9	ROS 2 Integration (Project Layout and Status)	19
9.1	Current source-verified command and conversion definitions	19
9.2	Package layout	19
9.3	Joint B's configuration block	19
9.4	Current CAN checksum implementation	20
9.5	Building and running	20
9.6	Recommended next steps toward full ROS 2 control	20
10	Troubleshooting	21
11	Quick Reference	22

1 Introduction

This guide walks through everything needed to take a fresh Ubuntu virtual machine and a CANable 2 USB-to-CAN adapter, and end up reliably talking to one joint (“Joint B”) of an Arctos robotic arm over CAN bus, with a clear path toward integrating that control into the arm’s ROS 2 driver.

It is written for someone who has never worked with a CAN bus or ROS 2 before. Two ideas make everything else in this guide make sense:

1.1 CAN Bus, in one paragraph

A Controller Area Network (CAN) bus is a two-wire (`CAN_H` / `CAN_L`) differential serial network. Every device on the bus — in this project, every joint’s motor controller — listens to every message and only acts on the ones addressed to its own ID. Your computer does not talk CAN natively; the CANable 2 adapter converts USB traffic into real CAN electrical signals, and Linux’s SocketCAN subsystem makes that adapter appear as an ordinary network interface (`can0`) that command-line tools and Python can use directly.

1.2 Gearboxes and Closed-Loop Steppers, in one paragraph

The motor at each joint does not drive the joint directly. It spins through a heavy reduction gearbox — for Joint B, a **67.82:1** ratio — so the motor shaft must turn 67.82 times to rotate the physical joint once. The motor itself is a closed-loop NEMA 17 stepper: unlike a plain stepper, it has a magnetic encoder on the back of the shaft, so the MKS Servo42D controller always knows the true shaft position and can correct for missed steps. If the joint is obstructed and the shaft falls too far behind its commanded target, the controller trips a *tracking-error protection stall* and locks the motor down; recovering from that requires power-cycling the controller.

Caution: Read this before touching real hardware

Robotic arm joints, even small hobby ones, carry enough torque to pinch fingers or damage the linkage if commanded incorrectly. Keep hands clear of belts, pulleys, and linkages whenever the motor power supply is live, keep `candump` running in a separate terminal at all times, and know your disable/e-stop commands (Section 7) before you send your first motion command.

2 Hardware Overview

2.1 Joint / CAN ID Map

Joint	Physical Role	CAN ID (hex)	Bitrate
Joint A	Elbow	0x04	500 kbit/s
Joint C	Wrist joint	0x06	500 kbit/s
Joint B	Wrist rotation	0x05	500 kbit/s

Table 1: Arctos CAN node map. Only Joint B (0x05) has been bench-tested in this guide; A and C are included for reference and use the same protocol.

2.2 Joint B Motor / Controller Specifications

Parameter	Value
Motor	NEMA 17, closed-loop stepper
Motor body length	23 mm
Holding torque	24 N·cm
Rated current	1.2 A
Shaft diameter	5 mm
Motor step angle	1.8° (200 full steps/revolution)
Controller	MKS Servo42D
Encoder resolution	16,384 counts/revolution (14-bit)
Joint gear ratio	67.82:1
Joint direction	Inverted (<code>inverted: true</code> in config)

Table 2: Joint B hardware parameters, confirmed from `arctos_controller.yaml` and the MKS datasheet-equivalent panel settings.

Caution: Do not confuse motor steps with encoder counts

The stepper’s 200 full-steps/revolution is a physical stepping resolution. It is *not* the same number as the encoder’s 16,384 counts/revolution, which is what all CAN position math in this guide uses. Mixing the two up is a common source of a 80× scaling error.

2.3 CANable 2 Adapter

- USB VID:PID 16d0:117e
- Contains its own CAN controller hardware — **do not** add an external MCP2515 module in series with it.
- Only three wires are needed between the adapter and the MKS controller: `CAN_H`, `CAN_L`, and `GND`. Do *not* connect the CANable’s own power lines to the motor controller.

2.4 Wiring Diagram

```
1 Ubuntu Virtual Machine
2   |
3   USB
4   |
5   CANable 2
6   |
7   CAN_H / CAN_L / GND
8   |
9   MKS Servo42D Controller
10  |
11  Joint Motor
```

Listing 1: Physical signal path

2.5 Bus Termination Check (power off)

With the system powered off, measure resistance between `CAN_H` and `CAN_L`:

Reading	Meaning
$\approx 60\Omega$	Correct — two 120Ω terminators in parallel, wiring is likely fine
$\approx 120\Omega$	Only one terminator present
Infinite / open	Wiring is disconnected
Very low	Possible short

3 Bringing Up the CAN Interface on Ubuntu

Every time the CANable adapter is plugged in (including after a VirtualBox USB reattach), the interface must be rebuilt from scratch. This is the full sequence.

3.1 Step 1: Confirm the USB device is visible

```
1 lsusb
2 # Expected line:
3 #   ID 16d0:117e CANable2
```

If using VirtualBox, also confirm the USB device is attached to the Ubuntu VM in the VirtualBox USB device menu, not just plugged into the host.

3.2 Step 2: Find the serial device

```
1 ls -l /dev/ttyACM*
2 # Example: /dev/ttyACM0 (the number can change after every reconnect --
3 # always re-check this before starting CAN)
```

3.3 Step 3: Load the CAN driver if needed

```
1 lsmod | grep gs_usb
2 # If nothing prints:
3 sudo modprobe gs_usb
4 sudo dmesg | tail -30
5 # Look for: "registered new interface driver gs_usb"
```

3.4 Step 4: Clean up any old session

```
1 sudo pkill slcand
2 sudo ip link delete can0 2>/dev/null
```

3.5 Step 5: Bind the adapter to a CAN interface

Caution: The slcand bitrate flag is a common mistake

slcand's `-s` flag selects the bitrate by a fixed code, *not* the raw number: `-s0=10k`, `-s1=20k`, `-s2=50k`, `-s3=100k`, `-s4=125k`, `-s5=250k`, `-s6=500k`, `-s7=800k`, `-s8=1M`. Joint B is configured for **500 kbit/s**, so the correct flag is `-s6`. Earlier debugging logs for this project inconsistently used `-s8` (which actually requests 1 Mbit/s) in some sessions — if a bitrate mismatch happens, `candump` will show your own outgoing frames but the motor will never respond (see Section 10). Always double-check this flag, or set the bitrate explicitly in Step 6 below to be safe either way.

```
1 sudo slcand -o -c -s6 /dev/ttyACM0 can0
```

3.6 Step 6: Bring the link up at the correct bitrate

Setting the bitrate explicitly on the `ip link` command as well removes any ambiguity from the `slcand` flag:

```
1 sudo ip link set can0 up type can bitrate 500000
```

3.7 Step 7: Verify

```
1 ip -details -statistics link show can0
```

Look for:

```
1 can0: <NOARP,UP,LOWER_UP> ...
2 bitrate 500000
3 can state ERROR-ACTIVE
```

RX packets should increase whenever the motor responds to something; TX packets increase whenever you send. Bus errors climbing, or a bus-off state, both point back to wiring or bitrate.

3.8 Shutdown procedure

Before unplugging the CANable adapter:

```
1 # 1. Stop any running Python control script: Ctrl+C
2 # 2. Bring the interface down
3 sudo ip link set can0 down
4 # 3. Stop slcand
5 sudo pkill slcand
6 # 4. Then physically unplug the adapter
```

4 MKS Servo42D Panel Configuration

On the controller itself (Joint B), confirm these settings before attempting CAN communication:

Setting	Value
CAN ID	05
CAN Rate	500 (kbit/s)
CAN Response (CANRSP)	ENABLE
CAN CRC	SUM-8
Working Mode	CR_CLOSE (see caution below)

Save settings and power-cycle the controller after any change. The CAN ID in particular must actually be committed, not just visible in the menu.

Caution: Working-mode mismatch is a leading suspect for “command acknowledged, motor doesn’t move”

The Arctos ROS 2 driver source defines six working modes and defaults to `working_mode = 2`, i.e. `CR_vFOC`:

```
1 enum class MotorMode {
2     CR_OPEN   = 0,
3     CR_CLOSE  = 1,
4     CR_vFOC   = 2,
5     SR_OPEN   = 3,
6     SR_CLOSE  = 4,
7     SR_vFOC   = 5
8 };
```

If the physical panel is set to `CR_CLOSE` while a motion command was generated assuming `CR_vFOC` (or vice versa), the controller can accept and checksum-acknowledge a motion frame without actually rotating the shaft — which matches symptoms observed during testing (motor made a sound but did not visibly rotate). **Confirm the panel’s actual working mode and the value used when constructing 0x82 (SET_WORKING_MODE) frames before trusting any “no motion” or “no response” result as a wiring/CAN problem.**

5 The MKS CAN Protocol as Used by Arctos

5.1 Frame Structure and Checksum

Every command frame sent to Joint B follows the same pattern:

[command byte] [data bytes...] [checksum byte]

The checksum has been independently verified (matched by hand against multiple successful transmissions) as a simple 8-bit sum:

```
1 def checksum(motor_id, data_bytes):
2     return (motor_id + sum(data_bytes)) & 0xFF
```

Frame (no checksum)	Arithmetic	Checksum
31	0x31+0x05	36
3A	0x3A+0x05	3F
F1	0xF1+0x05	F6
F3 01	0xF3+0x01+0x05	F9
F5 00 64 14 00 0C E4	sum + 0x05	62

Table 3: Worked checksum examples for CAN ID 0x05 (Joint B). All verified by hand and against actual bus traffic.

Caution: Never hand-invent a checksum

Manually sent `cansend` commands with an incorrect or omitted checksum byte were the direct cause of several “the motor ignored my command” failures earlier in this project. Always compute the checksum from the formula above, never guess it.

5.2 Command Reference

Name	Byte	Notes
QUERY_MOTOR	0xF1	Confirms the controller is alive. Verified response: F1 01 F7.
READ_ENCODER (carry form)	0x30	See Section 5.4.
READ_ENCODER (alt. form)	0x31	Different response layout observed — unverified, see Section 5.4.
READ_VELOCITY	0x32	Response layout not yet confirmed; log raw bytes only.
READ_ERROR	0x39	Response layout not yet confirmed.
READ_ENABLE_STATE	0x3A	Response byte-to-state mapping not yet confirmed.
READ_SHAFT_PROTECTION_STATE	0x3E	Indicates a tracking-error stall.
SET_ENABLE_SETTINGS	0x85	Configuration, not a runtime enable — do not confuse with 0xF3.
SET_WORKING_MODE	0x82	See working-mode caution above.
ENABLE_MOTOR	0xF3	Payload 0x01=enable, 0x00=disable. Confirmed working both directions.
RELATIVE_POSITION	0xF4	See Section 5.5.
ABSOLUTE_POSITION	0xF5	See Section 5.5.
SPEED_CONTROL	0xF6	Not exercised in testing so far.

Name	Byte	Notes
EMERGENCY_STOP	0xF7	See Section 7.
POSITION_CONTROL	0xFD	Not exercised in testing so far.
ABSOLUTE_POSITION_PULSE	0xFE	Not exercised in testing so far.

Table 4: Command bytes confirmed from `motor_types.hpp`. “Not yet confirmed” items should be logged with raw `candump` output and cross-checked against source before writing parsing code.

5.3 Enable / Disable / Query

```

1 # Enable
2 cansend can0 005#F301F9
3
4 # Disable
5 cansend can0 005#F300F8
6
7 # Query (liveness check)
8 cansend can0 005#F1F6
9 # Expected response: 005#F1 01 F7

```

Listing 2: Enable, disable, and query – all checksum-verified

5.4 Reading the Encoder

The current ROS 2 source explicitly defines `READ_ENCODER = 0x31` in `motor_types.hpp`. The receive path in `motor_driver.cpp` sends the six bytes following the `0x31` command byte to `CANProtocol::decodeInt`. Therefore, for the current driver, **0x31 is the encoder-position command used for position math.**

```

1 cansend can0 005#3136
2 # Observed response:
3 # can0 005 [8] 31 FF FF FF FF FF F7 28
4
5 cansend can0 005#3136
6 # Another observed response:
7 # can0 005 [8] 31 00 00 00 00 00 01 37

```

Listing 3: Observed Joint B encoder traffic

The current C++ decoder requires exactly six payload bytes after the command: bytes 1–6 of the CAN frame are combined as one signed 48-bit big-endian integer. The decoder then converts that value to degrees using

```

1 int64_t addition_value =
2     (static_cast<int64_t>(data[0]) << 40) |
3     (static_cast<int64_t>(data[1]) << 32) |
4     (static_cast<int64_t>(data[2]) << 24) |
5     (static_cast<int64_t>(data[3]) << 16) |
6     (static_cast<int64_t>(data[4]) << 8) |
7     static_cast<int64_t>(data[5]);
8
9 if (addition_value & 0x800000000000) {

```

```

10     addition_value |= 0xFFFF000000000000;
11 }
12
13 double motor_angle_deg =
14     (addition_value * MotorConstants::DEGREES_PER_REVOLUTION)
15     / MotorConstants::ENCODER_STEPS;

```

The driver's unit tests provide the intended scale: a six-byte value of 00 00 01 00 00 00 must decode to 360°, and 00 00 00 00 20 00 must decode to 180°. Thus the source currently treats 0x4000 = 16384 encoder counts as one motor-shaft revolution.

Caution: Do not mix the three observed read commands

The current source defines READ_ENCODER = 0x31, READ_PULSES = 0x33, and READ_RAW_ENCODER = 0x35 as different commands. Captured responses include:

```

1 can0 005 [8] 31 00 00 00 00 01 37
2 can0 005 [6] 33 00 00 00 23 5B
3 can0 005 [8] 35 00 00 00 00 0B 62 A7

```

Only the 0x31 response is passed through decodeInt48() by the current ROS 2 driver. The exact numerical meaning of the 0x33 and 0x35 payloads has not been established from the source examined so far, so they should not be substituted into the 0x31 position formula.

5.5 Motion Commands

The Arctos driver's absolute-position frame construction, confirmed directly from motor_driver.cpp:

```

1 double motor_position = position * joint.gear_ratio;
2 double motor_position_deg = motor_position * MotorConstants::RAD_TO_DEG;
3 int32_t encoder_counts = static_cast<int32_t>(
4     (motor_position_deg * MotorConstants::ENCODER_STEPS)
5     / MotorConstants::DEGREES_PER_REVOLUTION
6 );
7
8 std::vector<uint8_t> data = {
9     CANCommands::ABSOLUTE_POSITION,                                // 0xF5
10    static_cast<uint8_t>((speed >> 8) & 0xFF),
11    static_cast<uint8_t>(speed & 0xFF),
12    acc_value,
13    static_cast<uint8_t>((encoder_counts >> 16) & 0xFF),
14    static_cast<uint8_t>((encoder_counts >> 8) & 0xFF),
15    static_cast<uint8_t>(encoder_counts & 0xFF)
16 };

```

Frame layout: [0xF5] [speed_hi] [speed_lo] [accel] [pos_hi] [pos_mid] [pos_lo] [checksum]. The 3-byte position field is wide enough (24 bits, up to ≈16.7 million) to carry a full multi-turn absolute count directly – it is *not* limited to one revolution.

Caution: Absolute-vs-relative behavior of 0xF5 is not fully confirmed

Two bench tests sent structurally valid, correctly checksummed 0xF5 frames and produced results that do not cleanly agree with a single interpretation: one test's resulting motion was closer

to what a *relative delta* would produce, the next test's was not. Until this is resolved by reading `can_protocol.cpp`'s `sendFrame()` implementation and comparing it against controlled single-variable tests, treat any 0xF5 target as **unverified**, and always keep the commanded magnitude small enough that either interpretation is safe (Section 8).

6 From Raw Encoder Counts to Joint Degrees

Combining the confirmed encoder resolution and gear ratio:

$$\text{counts per Joint-B degree} = \frac{\text{gear_ratio} \times \text{ENCODER_STEPS}}{360} = \frac{67.82 \times 16384}{360} \approx 3086.56$$

Joint B motion	Raw encoder counts (approx.)
0.01°	31
0.1°	309
1°	3087
360° (one full joint revolution)	1,111,163

Table 5: Worked conversions using the verified 67.82:1 gear ratio and 16,384-count encoder.

Caution: Two other conversion numbers exist in this project’s history – do not use them

An early calibration pass, done before the gear ratio was discovered in `arctos_controller.yaml`, assumed the motor’s raw shaft rotation *was* the joint rotation and produced a figure of “≈3300 counts per 90°.” That number is real data, but it describes the *motor shaft*, not the joint – once the 67.82:1 reduction is applied, that same 3300-count motion corresponds to only ≈ 1.07° of actual Joint B travel. Separately, a since-superseded Python calibration note referenced `COUNTS_PER_REV = 16050`. That value is not used by the current C++ driver. The current source explicitly defines `ENCODER_STEPS = 16384` and the unit tests are written around that scale. For calculations that reproduce the current ROS 2 driver’s command math, use the source-defined 16384 value. The 16050 value should be retained as historical empirical data, not silently substituted into the C++ formula.

Caution: A conflicting “–813 pulses/degree” figure was also found – do not use it

A separate driver draft (`mks_driver.py`) defined `pulses_per_degree = -813.0` for Joint B and used it directly as the value written into the CAN frame’s position field. Its origin has not yet been reconciled with the current ROS 2 source:

```
1 target_pulses = int(degrees * profile["pulses_per_degree"])
2 estimated_final_encoder = current_encoder + int(target_pulses * -0.947)
3 MIN_LIMIT, MAX_LIMIT = -73000, 73000
4 if estimated_final_encoder < MIN_LIMIT or estimated_final_encoder > MAX_LIMIT:
5     print("Movement Blocked...")
6     return
7 pos = target_pulses & 0xFFFFFFF # <- this is what is actually transmitted
```

This does not match the source-verified ≈ 3086.56 counts/degree figure – it is off by roughly a factor of 3.8. More importantly, the safety-limit *estimate* (`target_pulses * -0.947`) uses a completely different multiplier than the value that is actually transmitted on the bus (`target_pulses` directly). Because the two don’t agree, the “Safety Workspace Cage” cannot reliably predict where a command will actually send the joint, and so cannot be trusted to block an unsafe one. **Do not use this**

conversion constant or this safety-limit code until it is reconciled against the verified gear-ratio math above.

7 Safety Commands

Action	Command	Notes
Disable motor	<code>cansend can0 005#F300F8</code>	Checksum-verified; use this form.
Enable motor	<code>cansend can0 005#F301F9</code>	Checksum-verified.
Emergency stop	<code>cansend can0 005#F7FC</code>	Verify before relying on it – see caution below.

Caution: Confirm the emergency-stop frame before you need it in an emergency

Notes from this project show the emergency-stop command sent as a bare F7 with no checksum byte at all, which is inconsistent with every other verified command on this bus (all of which required a checksum, $0xF7+0x05=0xFC$ by the same formula used everywhere else). Do not assume either form works under load. With the joint mechanically unloaded and clear of obstructions, deliberately trigger both forms and confirm with `candump` that the motor actually loses holding torque before you depend on either one during a real motion test.

8 A Safe First Motion Test

Given the open questions in Sections 5.5 and above, the right first motion test is one that is safe *regardless of which interpretation turns out to be correct*. A magnitude of ≈ 300 raw counts ($\approx 0.1^\circ$ of Joint B) satisfies this: if the firmware treats the field as an absolute target, the worst case is a small jump near zero; if it treats it as a relative delta, it is simply a 0.1° nudge.

8.1 Procedure

1. Bring up `can0` per Section 3.
2. Open a second terminal and leave `candump -tz can0` running.
3. Confirm the joint is mechanically clear.
4. Run the script below. It reads the true absolute position (carry-aware), pauses for confirmation, sends one small move, re-reads the position, and reports the actual counts and degrees moved – which also tells you empirically whether `0xF5` is behaving as absolute or relative.
5. The motor is disabled in a `finally` block regardless of how the script exits.

8.2 Script: `joint_b_micro_move_test.py`

```
1 #!/usr/bin/env python3
2  """
3  Safe, small (~0.1 deg) test move for Arctos Joint B (MKS Servo42D, CAN ID 0x05).
4  Reads the FULL absolute encoder (carry * 16384 + value) before and after,
5  uses a magnitude small enough to be safe under either an absolute or
6  relative interpretation of the 0xF5 position field, and always disables
7  the motor before exiting.
8  """
9  import can
10 import time
11
12 CAN_ID = 0x05
13 CHANNEL = "can0"
14 BUSTYPE = "socketcan"
15
16 TEST_DELTA_COUNTS = 300          # ~0.1 degree of Joint B (67.82:1 gear, 16384 cpr)
17 SPEED = 20                      # low speed for safety
18 ACCEL = 5                       # low acceleration for safety
19 COUNTS_PER_DEGREE = 67.82 * 16384 / 360.0  # ~3086.56
20
21
22 def checksum(motor_id, data_bytes):
23     return (motor_id + sum(data_bytes)) & 0xFF
24
25
26 def send_frame(bus, motor_id, data_bytes):
27     crc = checksum(motor_id, data_bytes)
28     frame = data_bytes + [crc]
29     msg = can.Message(arbitration_id=motor_id, data=frame, is_extended_id=False)
30     bus.send(msg)
31     return frame
32
33
```

```

34 def read_encoder(bus, motor_id, timeout=1.0):
35     send_frame(bus, motor_id, [0x30])
36     end = time.time() + timeout
37     while time.time() < end:
38         resp = bus.recv(timeout=end - time.time())
39         if resp is None:
40             break
41         if resp.arbitration_id == motor_id and len(resp.data) >= 7 and resp.data
[0] == 0x30:
42             carry = int.from_bytes(resp.data[1:5], byteorder="big", signed=True)
43             value = int.from_bytes(resp.data[5:7], byteorder="big", signed=False)
44             return carry, value, carry * 16384 + value
45         raise TimeoutError("No encoder response received")
46
47
48 def enable_motor(bus, motor_id):
49     send_frame(bus, motor_id, [0xF3, 0x01]); time.sleep(0.2)
50
51
52 def disable_motor(bus, motor_id):
53     send_frame(bus, motor_id, [0xF3, 0x00]); time.sleep(0.2)
54
55
56 def move_small(bus, motor_id, delta_counts, speed, accel):
57     speed_hi, speed_lo = (speed >> 8) & 0xFF, speed & 0xFF
58     pos = delta_counts & 0xFFFF
59     pos_hi, pos_mid, pos_lo = (pos >> 16) & 0xFF, (pos >> 8) & 0xFF, pos & 0xFF
60     data = [0xF5, speed_hi, speed_lo, accel, pos_hi, pos_mid, pos_lo]
61     return send_frame(bus, motor_id, data)
62
63
64 def main():
65     bus = can.interface.Bus(channel=CHANNEL, bustype=BUSTYPE)
66     try:
67         carry0, value0, abs0 = read_encoder(bus, CAN_ID)
68         print(f"Start: carry={carry0} value={value0} absolute={abs0}")
69
70         input("Enable motor and send a ~0.1 degree test move? "
71              "Press Enter to continue, Ctrl+C to abort...")
72
73         enable_motor(bus, CAN_ID)
74         frame = move_small(bus, CAN_ID, TEST_DELTA_COUNTS, SPEED, ACCEL)
75         print(f"Sent frame: {[hex(b) for b in frame]}")
76         time.sleep(2.0)
77
78         carry1, value1, abs1 = read_encoder(bus, CAN_ID)
79         moved = abs1 - abs0
80         print(f"End:   carry={carry1} value={value1} absolute={abs1}")
81         print(f"Observed change: {moved} raw counts "
82              f"({moved / COUNTS_PER_DEGREE:.4f} deg of Joint B)")
83         print("moved ~= 300      -> field behaves as a RELATIVE delta")
84         print("moved ~= 300-abs0 -> field behaves as an ABSOLUTE target")
85     finally:
86         print("Disabling motor.")
87         disable_motor(bus, CAN_ID)
88         bus.shutdown()
89
90
91 if __name__ == "__main__":

```


9 ROS 2 Integration (Project Layout and Status)

The confirmed working CAN-level control (Sections 3–8) is the foundation for the ROS 2 driver package. As of this guide, **full closed-loop joint control through ROS 2 has not yet been demonstrated** – only direct, script-level CAN control has. The pieces below are the confirmed structure to build on.

9.1 Current source-verified command and conversion definitions

The current `motor_types.hpp` defines the following relevant values:

```
1 READ_ENCODER          = 0x31
2 READ_PULSES           = 0x33
3 READ_RAW_ENCODER      = 0x35
4 SET_WORKING_MODE      = 0x82
5 SET_SUBDIVISIONS      = 0x84
6 SET_ENABLE_SETTINGS   = 0x85
7
8 ENABLE_MOTOR          = 0xF3
9 RELATIVE_POSITION     = 0xF4
10 ABSOLUTE_POSITION    = 0xF5
11 POSITION_CONTROL      = 0xFD
12 ABSOLUTE_POSITION_PULSE = 0xFE
13
14 MotorParameters::working_mode = 2
15 MotorParameters::subdivisions = 16
16
17 MotorConstants::ENCODER_STEPS = 16384.0
```

The mode enumeration identifies mode 2 as CR_vFOC, while the default `MotorParameters::working_mode` is also 2. This documents what the current code assumes; it does *not* establish that the physical motor panel is currently configured to that mode.

9.2 Package layout

```
1 ~/arctos_ws/src/ros2_arctos/arctos_motor_driver/
2   include/arctos_motor_driver/motor_types.hpp    # command byte + mode enums
3   src/motor_driver.cpp                          # frame construction, gear-ratio
4   math
5   src/can_protocol.cpp                          # sendFrame() / checksum
6 ~/arctos_ws/src/ros2_arctos/arctos_description/config/arctos_controller.yaml
7 # per-joint gear_ratio, inverted, zero_position, home_position, opposite_limit
```

9.3 Joint B's configuration block

```
1 B_joint:
2   motor_id: 5
3   requires_homing: false
4   inverted: true
5   gear_ratio: 67.82
6   hardware_type: MKS_42D
7   zero_position: -0.43627526433400465    # radians
8   home_position: 0.22467435360166688    # radians
9   opposite_limit: -0.4228851636789132    # radians
```

These are joint-space parameters in radians, not raw encoder counts – do not substitute a raw encoder reading for one of these without going through the gear-ratio conversion in Section 5 of this guide.

9.4 Current CAN checksum implementation

The current `CANProtocol::sendFrame()` implementation sets the DLC to the command-data length plus one checksum byte, copies the command data into the CAN frame, and computes the checksum as the low eight bits of the sum of the motor ID and every command-data byte:

```
1 uint16_t crc = motor_id;
2 for (size_t i = 0; i < data.size(); i++) {
3     crc += data[i];
4 }
5 crc &= 0xFF;
6 msg->data[data.size()] = static_cast<uint8_t>(crc);
```

This matches the checksum convention used elsewhere in this guide for the frames examined so far.

9.5 Building and running

```
1 cd ~/arctos_ws
2 colcon build --packages-select arctos_motor_driver
3 source ~/arctos_ws/install/setup.bash
4
5 ros2 node list
6 ros2 topic list
7 ros2 topic echo /topic_name
8 ros2 service list
9 ros2 action list
```

9.6 Recommended next steps toward full ROS 2 control

1. Resolve the 0xF5 absolute-vs-relative question using controlled, single-variable motion tests.
2. Determine the exact semantics of the observed 0x33 and 0x35 responses before using either as a substitute for the 0x31 position response.
3. Trace the origin of the historical -813 pulses/degree value in Git history / older scripts. Do not use it merely because it appears in an older driver draft.
4. Confirm the physical motor panel's working mode against the source's default `working_mode=2`; the source definition alone does not prove the panel setting.
5. Only once single-joint CAN control is deterministic, wire it into the ROS 2 controller's command topic / action interface.

10 Troubleshooting

Symptom	Likely cause / fix
CANable disappears after unplugging	<code>/dev/ttyACM#</code> number changed. Re-run <code>lsusb</code> and <code>ls -l /dev/ttyACM*</code> , use the new number in <code>slcand</code> .
<code>can0</code> does not exist	Old <code>slcand</code> session still holding it. <code>sudo pkill slcand; sudo ip link delete can0</code> then recreate.
<code>candump</code> only shows your own outgoing frames	Adapter is transmitting but nothing replies. Check bitrate (Caution in Section 3), CAN ID, CANRSP=ENABLE, and CAN_H/CAN_L wiring.
MKS display is on, no CAN response at all	Confirm CAN ID, CAN rate, CANRSP, and working mode on the panel match what you are sending.
Command is checksummed correctly and acknowledged, but motor does not move	Suspect working-mode mismatch (Section 3) or an incorrect 0xF5/0xF4 interpretation (Section 5.5) before assuming a wiring fault.
Motor gets hot during repeated testing	Expected for a continuously-energized stepper – disable (F300F8) between tests and let it cool; do not leave it enabled unnecessarily.

Table 6: Common failure modes observed during Joint B bring-up.

11 Quick Reference

```
1 # --- Bring up CAN ---
2 lsusb
3 ls -l /dev/ttyACM*
4 sudo pkill slcand
5 sudo ip link delete can0 2>/dev/null
6 sudo slcand -o -c -s6 /dev/ttyACM0 can0
7 sudo ip link set can0 up type can bitrate 500000
8 ip -details -statistics link show can0
9
10 # --- Monitor ---
11 candump -tz can0
12
13 # --- Query / read (all checksum-verified) ---
14 cansend can0 005#F1F6      # query motor      -> F1 01 F7
15 cansend can0 005#3035      # read encoder      -> 30 <carry:4><value:2><crc>
16
17 # --- Enable / disable / stop ---
18 cansend can0 005#F301F9    # enable
19 cansend can0 005#F300F8    # disable
20 cansend can0 005#F7FC      # emergency stop (VERIFY before relying on it)
21
22 # --- Shutdown ---
23 sudo ip link set can0 down
24 sudo pkill slcand
```

Listing 4: Copy/paste reference for a normal session